

Laboratorio práctico — DeepBlue Rescue

Implementación completa de la capa de controladores REST

Duración máxima: 3 horas

Nivel: Básico → Intermedio

Tecnologías: Java 21, Spring Boot 4, Spring MVC, Bean Validation, JUnit 5, Mockito y MockMvc

Modalidad: Parejas

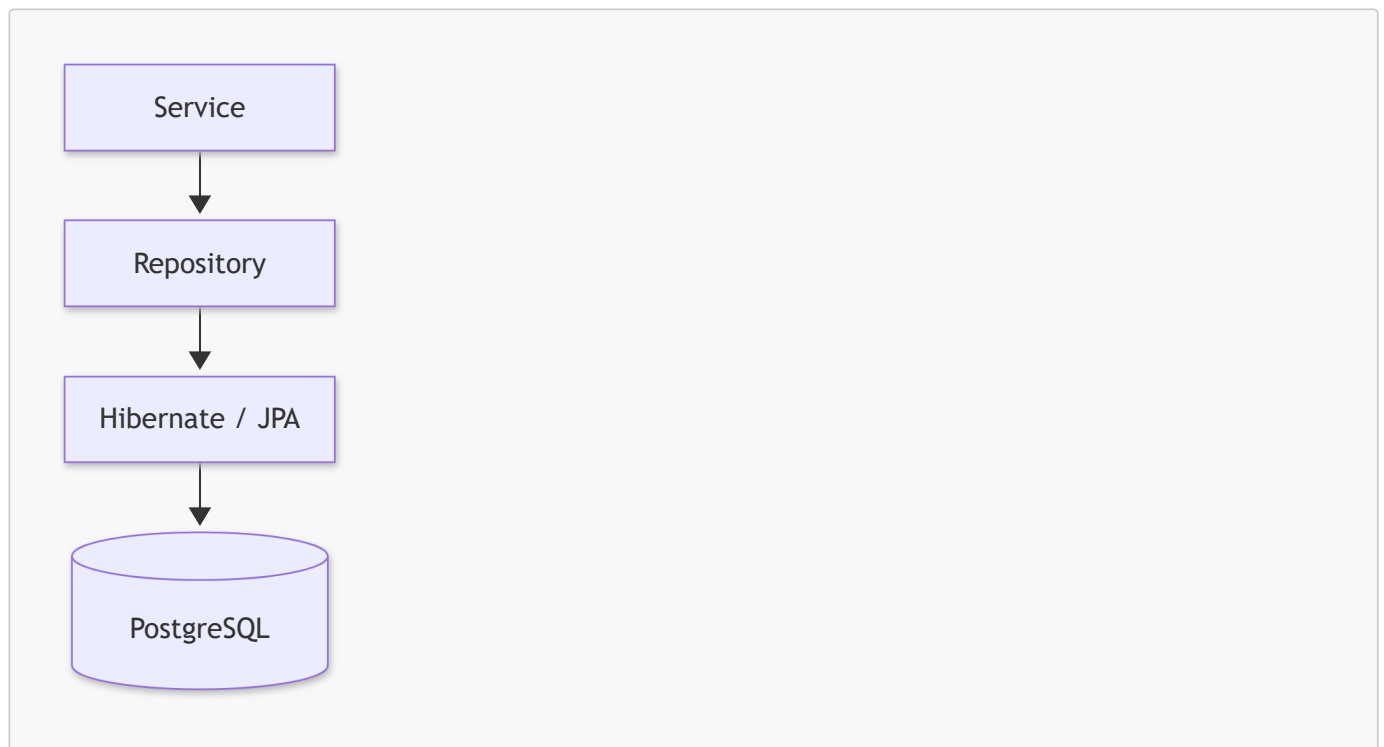
1. Contexto

DeepBlue Rescue es una plataforma para gestionar rescates, rehabilitación y tratamientos de fauna marina.

En laboratorios anteriores se implementaron:

- entidades JPA;
- relaciones;
- repositories;
- consultas;
- capa Service;
- DTOs;
- MapStruct;
- reglas de negocio;
- excepciones;
- pruebas unitarias de Service.

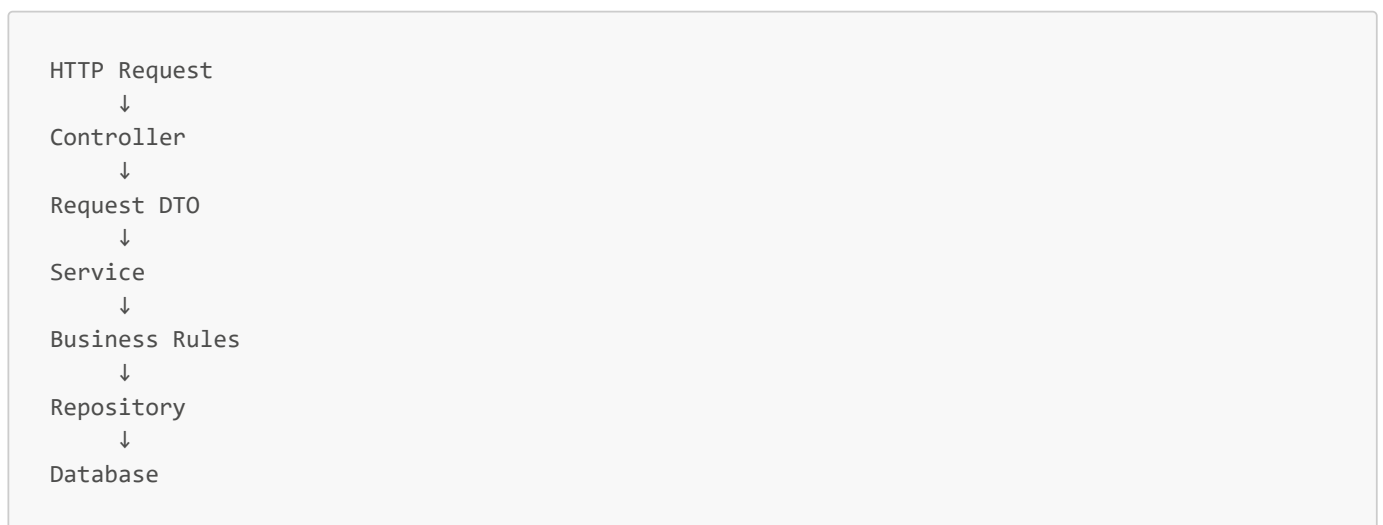
Hasta ahora la arquitectura puede representarse así:



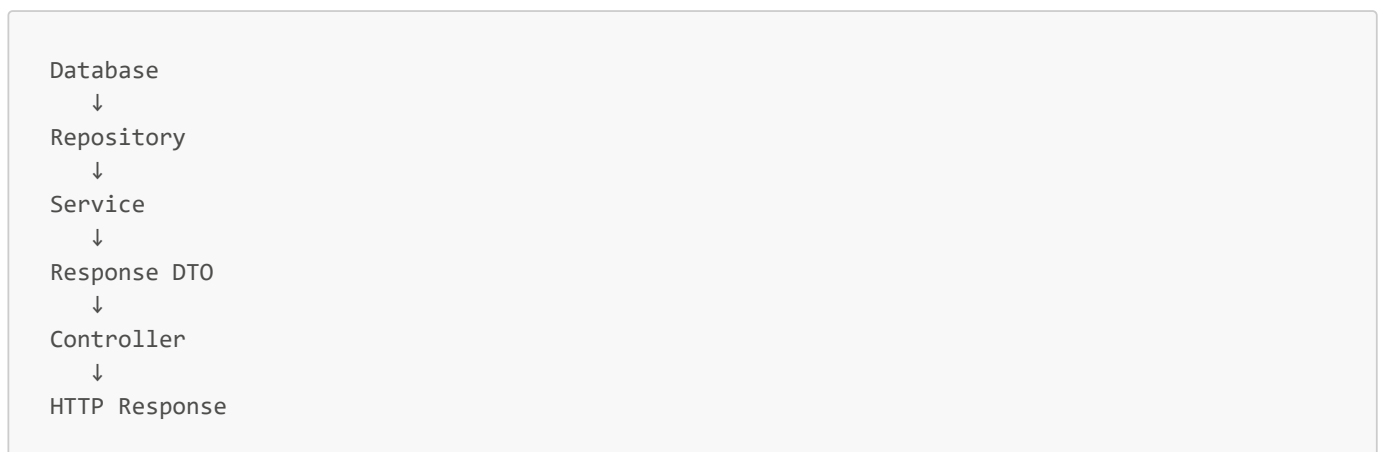
Ahora agregaremos la capa que permitirá exponer las operaciones mediante HTTP:



La aplicación evolucionará hacia:



La respuesta seguirá:



2. Objetivos de aprendizaje

Al finalizar el laboratorio, el estudiante deberá poder:

1. Explicar la responsabilidad de un Controller.
2. Crear Controllers REST utilizando `@RestController`.
3. Diseñar endpoints REST utilizando recursos.
4. Utilizar `@RequestMapping`.
5. Utilizar `@GetMapping`.
6. Utilizar `@PostMapping`.
7. Utilizar `@PatchMapping`.
8. Utilizar `@PathVariable`.
9. Utilizar `@RequestParam`.
10. Utilizar `@RequestBody`.
11. Aplicar `@Valid`.
12. Utilizar Bean Validation.
13. Diferenciar validación de entrada de reglas de negocio.
14. Utilizar `ResponseEntity`.
15. Retornar códigos HTTP apropiados.
16. Crear un contrato de error consistente mediante `ErrorResponse`.
17. Implementar `@RestControllerAdvice`.
18. Implementar `ResponseEntity<ErrorResponse>`.
19. Diferenciar errores 400, 404, 409 y 500.
20. Manejar errores de JSON inválido.
21. Manejar query parameters inválidos.
22. Probar Controllers mediante `@WebMvcTest`.
23. Reemplazar Services mediante `@MockitoBean`.
24. Utilizar `MockMvc`.
25. Utilizar `jsonPath`.
26. Verificar interacciones con Mockito.
27. Utilizar `verify(..., never())`.
28. Exponer todos los métodos definidos en la capa Service.
29. Evitar lógica de negocio en Controllers.
30. Diferenciar tests de Repository, Service y Controller.

3. Responsabilidad del Controller

El Controller debe encargarse de:

```
HTTP
URLs
HTTP Methods
Request Body
Path Variables
Query Parameters
Validation
Status Codes
Response Body
```

No debe encargarse de:

SQL
Repositories directamente
transacciones
reglas de negocio
cálculos de negocio
transiciones de estado

4. Separación de responsabilidades



Controller

Pregunta:

¿Cómo llega la petición?

Service

Pregunta:

¿Está permitida la operación?

Repository

Pregunta:

¿Cómo accedo a los datos?

5. Ejemplo incorrecto

Evitar:

```
@RestController
public class TreatmentController {

    private final TreatmentRepository repository;

    @PostMapping("/treatments")
    public Treatment create(...) {

        if (!specialist.isActive()) {
            throw new RuntimeException();
        }

        return repository.save(...);
    }
}
```

Problemas:

- Controller conoce Repository
- Controller conoce reglas del negocio
- Controller retorna Entity
- Controller se vuelve difícil de probar

6. Ejemplo correcto

```
@RestController
@RequestMapping("/api/treatments")
public class TreatmentController {

    private final TreatmentService service;

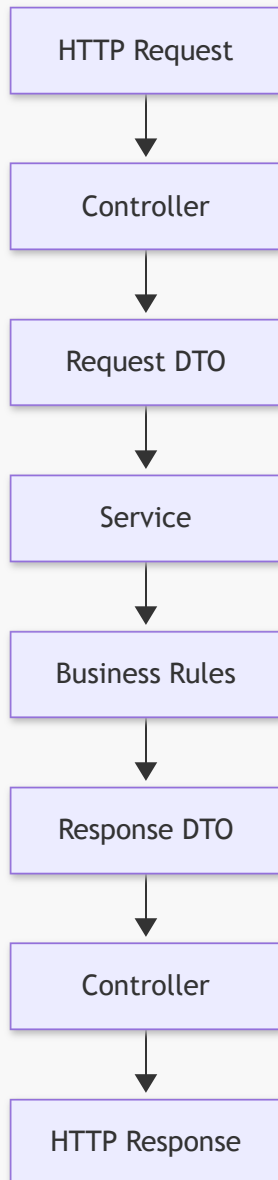
    public TreatmentController(
        TreatmentService service) {

        this.service = service;
    }
}
```

El Controller delegará:

```
service.register(request);
```

7. Modelo mental



PARTE I — PREPARACIÓN

8. Dependencias

Verificar en `pom.xml`:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

Bean Validation:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Testing:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

9. Estructura de paquetes

Agregar:

```
src/main/java/com/deepblue/rescue
├── controller
│   ├── RescueCaseController.java
│   ├── AnimalController.java
│   └── TreatmentController.java
├── dto
│   ├── request
│   │   ├── ChangeRescueStatusRequest.java
│   │   └── CreateTreatmentRequest.java
│   └── response
│       ├── RescueCaseResponse.java
│       ├── AnimalResponse.java
│       ├── TreatmentResponse.java
│       ├── TreatmentEligibilityResponse.java
│       └── ErrorResponse.java
├── exception
│   ├── ResourceNotFoundException.java
│   ├── BusinessException.java
│   └── GlobalExceptionHandler.java
├── service
├── repository
└── domain
```

Tests:

```
src/test/java/com/deepblue/rescue/controller
├── RescueCaseControllerTest.java
├── AnimalControllerTest.java
└── TreatmentControllerTest.java
```

PARTE II — CONTRATOS DE SERVICE

10. RescueCaseService

```
public interface RescueCaseService {

    RescueCaseResponse findByCode(
        String caseCode
    );

    List<RescueCaseResponse> findByStatus(
        RescueStatus status
    );

    RescueCaseResponse changeStatus(
        String caseCode,
        ChangeRescueStatusRequest request
    );
}
```

11. TreatmentService

```
public interface TreatmentService {

    TreatmentResponse register(
        CreateTreatmentRequest request
    );

    List<TreatmentResponse> findByAnimalCode(
        String animalCode
    );
}
```

12. AnimalService

```
public interface AnimalService {

    AnimalResponse findByCode(
```

```
        String animalCode
    );

    List<AnimalResponse>
    findAnimalsInRehabilitation();

    boolean canReceiveTreatment(
        String animalCode
    );
}
```

13. Cobertura de lógica de negocio

Todos los métodos anteriores deben quedar expuestos.

Service	Método	Endpoint
RescueCaseService	<code>findByCode()</code>	GET <code>/api/rescue-cases/{caseCode}</code>
RescueCaseService	<code>findByStatus()</code>	GET <code>/api/rescue-cases?status=...</code>
RescueCaseService	<code>changeStatus()</code>	PATCH <code>/api/rescue-cases/{caseCode}/status</code>
TreatmentService	<code>register()</code>	POST <code>/api/treatments</code>
TreatmentService	<code>findByAnimalCode()</code>	GET <code>/api/animals/{animalCode}/treatments</code>
AnimalService	<code>findByCode()</code>	GET <code>/api/animals/{animalCode}</code>
AnimalService	<code>findAnimalsInRehabilitation()</code>	GET <code>/api/animals/in-rehabilitation</code>
AnimalService	<code>canReceiveTreatment()</code>	GET <code>/api/animals/{animalCode}/treatment-eligibility</code>

Resultado:

```
8 métodos Service
  ↓
8 operaciones HTTP
```

PARTE III — DISEÑO DE LA API

14. Endpoints

Método	Endpoint	Operación
GET	<code>/api/rescue-cases/{caseCode}</code>	Consultar caso
GET	<code>/api/rescue-cases?status=...</code>	Buscar casos por estado
PATCH	<code>/api/rescue-cases/{caseCode}/status</code>	Cambiar estado
GET	<code>/api/animals/{animalCode}</code>	Consultar animal

Método	Endpoint	Operación
GET	<code>/api/animals/in-rehabilitation</code>	Animales en rehabilitación
GET	<code>/api/animals/{animalCode}/treatments</code>	Tratamientos del animal
GET	<code>/api/animals/{animalCode}/treatment-eligibility</code>	Elegibilidad
POST	<code>/api/treatments</code>	Registrar tratamiento

15. GET

Utilizar para:

consultar recursos

Ejemplo:

GET `/api/animals/AN-001`

GET no debe modificar el estado del sistema.

16. POST

Utilizar para crear un nuevo recurso.

Ejemplo:

POST `/api/treatments`

17. PATCH

Utilizar para modificar parcialmente un recurso.

Ejemplo:

PATCH `/api/rescue-cases/RES-001/status`

Estamos modificando:

status

no reemplazando todo el `RescueCase`.

PARTE IV — VALIDACIÓN

18. ChangeRescueStatusRequest

```
public record ChangeRescueStatusRequest(  
    @NotNull(  
        message = "Status is required"  
    )  
    RescueStatus status  
  
) {  
}
```

19. CreateTreatmentRequest

```
public record CreateTreatmentRequest(  
    @NotBlank(  
        message = "Animal code is required"  
    )  
    String animalCode,  
  
    @NotBlank(  
        message = "Specialist code is required"  
    )  
    String specialistCode,  
  
    @NotNull(  
        message = "Treatment date is required"  
    )  
    @PastOrPresent(  
        message = "Treatment date cannot be in the future"  
    )  
    LocalDateTime performedAt,  
  
    @NotNull(  
        message = "Treatment type is required"  
    )  
    TreatmentType type,  
  
    @NotBlank(  
        message = "Description is required"  
    )  
    @Size(  
        min = 10,  
        max = 500,  
        message = "Description must contain between 10 and 500 characters"  
    )  
    String description  
)
```

```
) {  
}
```

Imports:

```
import jakarta.validation.constraints.NotBlank;  
import jakarta.validation.constraints.NotNull;  
import jakarta.validation.constraints.PastOrPresent;  
import jakarta.validation.constraints.Size;
```

20. Validación de entrada vs lógica de negocio

Validación de entrada

```
animalCode vacío  
specialistCode vacío  
status null  
description demasiado corta  
fecha futura
```

Corresponde a:

DTO + Bean Validation

Regla de negocio

```
animal inexistente  
especialista inactivo  
caso RELEASED  
transición inválida  
tratamiento anterior al rescate
```

Corresponde a:

Service

21. Checkpoint

Clasifica:

Regla	DTO/Controller o Service
animalCode vacío	_____

Regla	DTO/Controller o Service
<code>status == null</code>	_____
animal inexistente	_____
especialista inactivo	_____
<code>description > 500</code>	_____
caso RELEASED	_____
transición ADMITTED → RELEASED	_____

PARTE V — CONTRATO DE ERRORES

22. ErrorResponse

Crear:

```
package com.deepblue.rescue.dto.response;

import java.time.LocalDateTime;
import java.util.Map;

public record ErrorResponse(
    LocalDateTime timestamp,
    int status,
    String error,
    String message,
    Map<String, String> details
) {
}
```

23. Objetivo del contrato

Todos los errores deben tener la misma estructura.

Ejemplo:

```
{
  "timestamp": "2026-10-04T19:01:00",
  "status": 404,
  "error": "Not Found",
  "message": "Animal not found: AN-999",
  "details": {}
}
```

24. Significado

timestamp

Momento en que ocurrió el error.

```
LocalDateTime.now()
```

status

Código numérico:

```
400  
404  
409  
500
```

error

Descripción estándar:

```
Bad Request  
Not Found  
Conflict  
Internal Server Error
```

message

Descripción específica:

```
Animal not found: AN-999
```

details

Información adicional.

Ejemplo:

```
{  
  "animalCode": "Animal code is required",  
  "description": "Description is required"  
}
```

Sin detalles:

```
Map.of()
```

PARTE VI — RESCUE CASE CONTROLLER

25. Crear Controller

```
@RestController
@RequestMapping("/api/rescue-cases")
public class RescueCaseController {

    private final RescueCaseService service;

    public RescueCaseController(
        RescueCaseService service) {

        this.service = service;
    }
}
```

26. findByCode()

```
@GetMapping("/{caseCode}")
public ResponseEntity<RescueCaseResponse>
findByCode(
    @PathVariable String caseCode) {

    return ResponseEntity.ok(
        service.findByCode(caseCode)
    );
}
```

Request:

```
GET /api/rescue-cases/RES-2026-001
```

Respuesta:

```
200 OK
```

Ejemplo:

```
{
  "id": 1,
  "caseCode": "RES-2026-001",
  "rescueDate": "2026-08-20",
  "rescueLocation": "Bahía Concha",
  "status": "IN_REHABILITATION",
  "centerCode": "DB-CAR",
}
```

```
"animalCode": "AN-2026-001"  
}
```

Utiliza:

```
RescueCaseService.findByCode()
```

27. @PathVariable

En:

```
/api/rescue-cases/RES-2026-001
```

Spring captura:

```
RES-2026-001
```

mediante:

```
@PathVariable String caseCode
```

28. findByStatus()

```
@GetMapping  
public ResponseEntity<List<RescueCaseResponse>>  
findByStatus(  
    @RequestParam RescueStatus status) {  
  
    return ResponseEntity.ok(  
        service.findByStatus(status)  
    );  
}
```

Request:

```
GET /api/rescue-cases?status=IN_REHABILITATION
```

Utiliza:

```
RescueCaseService.findByStatus()
```

29. @RequestParam

Spring convierte:

```
status=IN_REHABILITATION
```

a:

```
RescueStatus.IN_REHABILITATION
```

30. changeStatus()

```
@PatchMapping("/{caseCode}/status")
public ResponseEntity<RescueCaseResponse>
changeStatus(

    @PathVariable String caseCode,

    @Valid
    @RequestBody
    ChangeRescueStatusRequest request) {

    return ResponseEntity.ok(
        service.changeStatus(
            caseCode,
            request
        )
    );
}
```

Request:

```
PATCH /api/rescue-cases/RES-2026-001/status
Content-Type: application/json
```

```
{
  "status": "READY_FOR_RELEASE"
}
```

31. Flujo del PATCH



Syntax error in text mermaid version 12.1.0

Parse error on line 4: ...tatusRequest]C --> D[@Valid]D --> E[Co
-----^ Expecting 'AMP', 'COLON', 'PIPE', 'TESTSTR',
'DOWN', 'DEFAULT', 'NUM', 'COMMA', 'NODE_STRING', 'BRKT', 'MINUS', 'MULT',
'UNICODE_TEXT', got 'LINK_ID'

PARTE VII — TREATMENT CONTROLLER

32. Crear Controller

```
@RestController
@RequestMapping("/api/treatments")
public class TreatmentController {

    private final TreatmentService service;

    public TreatmentController(
        TreatmentService service) {

        this.service = service;
    }
}
```

33. register()

```
@PostMapping
public ResponseEntity<TreatmentResponse>
register(

    @Valid
    @RequestBody
    CreateTreatmentRequest request) {

    TreatmentResponse response =
        service.register(request);

    return ResponseEntity
        .status(HttpStatus.CREATED)
```

```
        .body(response);  
    }
```

Utiliza:

```
TreatmentService.register()
```

34. Request

```
POST /api/treatments  
Content-Type: application/json
```

```
{  
  "animalCode": "AN-2026-001",  
  "specialistCode": "SPEC-001",  
  "performedAt": "2026-08-21T09:30:00",  
  "type": "WOUND_CARE",  
  "description": "Cleaning and evaluation of left front flipper injury."  
}
```

35. Response

```
201 Created
```

```
{  
  "id": 100,  
  "animalCode": "AN-2026-001",  
  "specialistCode": "SPEC-001",  
  "performedAt": "2026-08-21T09:30:00",  
  "type": "WOUND_CARE",  
  "description": "Cleaning and evaluation of left front flipper injury."  
}
```

36. ¿Por qué 201?

Porque la operación creó un nuevo recurso.

```
POST  
↓  
nuevo Treatment
```



201 Created

PARTE VIII — ANIMAL CONTROLLER

37. Crear Controller

```
@RestController
@RequestMapping("/api/animals")
public class AnimalController {

    private final AnimalService animalService;

    private final TreatmentService
        treatmentService;

    public AnimalController(
        AnimalService animalService,
        TreatmentService treatmentService) {

        this.animalService = animalService;
        this.treatmentService = treatmentService;
    }
}
```

38. findByCode()

```
@GetMapping("/{animalCode}")
public ResponseEntity<AnimalResponse>
findByCode(
    @PathVariable String animalCode) {

    return ResponseEntity.ok(
        animalService.findByCode(animalCode)
    );
}
```

Utiliza:

```
AnimalService.findByCode()
```

39. findAnimalsInRehabilitation()

```
@GetMapping("/in-rehabilitation")
public ResponseEntity<List<AnimalResponse>>
findAnimalsInRehabilitation() {

    return ResponseEntity.ok(
        animalService
            .findAnimalsInRehabilitation()
    );
}
```

Request:

```
GET /api/animals/in-rehabilitation
```

Utiliza:

```
AnimalService.findAnimalsInRehabilitation()
```

40. findTreatments()

```
@GetMapping("/{animalCode}/treatments")
public ResponseEntity<List<TreatmentResponse>>
findTreatments(
    @PathVariable String animalCode) {

    return ResponseEntity.ok(
        treatmentService
            .findByAnimalCode(animalCode)
    );
}
```

Request:

```
GET /api/animals/AN-001/treatments
```

Utiliza:

```
TreatmentService.findByAnimalCode()
```

41. TreatmentEligibilityResponse

```
public record TreatmentEligibilityResponse(  
    String animalCode,  
    boolean eligible  
) {  
}
```

42. canReceiveTreatment()

```
@GetMapping(  
    "{animalCode}/treatment-eligibility"  
)  
public ResponseEntity<TreatmentEligibilityResponse>  
canReceiveTreatment(  
    @PathVariable String animalCode) {  
  
    boolean eligible =  
        animalService  
            .canReceiveTreatment(  
                animalCode  
            );  
  
    return ResponseEntity.ok(  
        new TreatmentEligibilityResponse(  
            animalCode,  
            eligible  
        )  
    );  
}
```

Request:

```
GET /api/animals/AN-001/treatment-eligibility
```

Response:

```
{  
  "animalCode": "AN-001",  
  "eligible": true  
}
```

43. Checkpoint de cobertura

```
RescueCaseService  
├─ findByCode          ✓  
├─ findByStatus       ✓  
└─ changeStatus       ✓
```

```
TreatmentService
├─ register          ✓
└─ findByAnimalCode  ✓

AnimalService
├─ findByCode        ✓
├─ findAnimalsInRehabilitation ✓
└─ canReceiveTreatment ✓
```

No debe quedar ningún método Service sin utilizar.

PARTE IX — HTTP STATUS

44. Códigos utilizados

Código	Uso
200	Consulta o actualización correcta
201	Recurso creado
400	Request inválido
404	Recurso inexistente
409	Conflicto con regla de negocio
500	Error inesperado

45. 400 Bad Request

Ejemplo:

```
{
  "animalCode": "",
  "specialistCode": "",
  "type": null,
  "description": ""
}
```

El request es inválido.

46. 404 Not Found

Ejemplo:

```
GET /api/animals/AN-999
```

Service lanza:

```
ResourceNotFoundException
```

Respuesta:

```
404 Not Found
```

47. 409 Conflict

Ejemplo:

```
Animal RELEASED  
+  
nuevo Treatment
```

El JSON puede ser válido, pero la operación viola una regla de negocio.

Respuesta:

```
409 Conflict
```

PARTE X — GLOBAL EXCEPTION HANDLER

48. Crear GlobalExceptionHandler

```
@RestControllerAdvice  
public class GlobalExceptionHandler {  
}
```

Todos sus métodos deben retornar:

```
ResponseEntity<ErrorResponse>
```

49. ResourceNotFoundException

```
@ExceptionHandler(  
    ResourceNotFoundException.class
```

```
)  
public ResponseEntity<ErrorResponse>  
handleResourceNotFound(  
    ResourceNotFoundException ex) {  
  
    HttpStatus status =  
        HttpStatus.NOT_FOUND;  
  
    ErrorResponse error =  
        new ErrorResponse(  
            LocalDateTime.now(),  
            status.value(),  
            status.getReasonPhrase(),  
            ex.getMessage(),  
            Map.of()  
        );  
  
    return ResponseEntity  
        .status(status)  
        .body(error);  
}
```

50. Ejemplo 404

```
{  
  "timestamp": "2026-10-04T19:01:00",  
  "status": 404,  
  "error": "Not Found",  
  "message": "Animal not found: AN-999",  
  "details": {}  
}
```

51. BusinessException

```
@ExceptionHandler(  
    BusinessException.class  
)  
public ResponseEntity<ErrorResponse>  
handleBusinessRule(  
    BusinessException ex) {  
  
    HttpStatus status =  
        HttpStatus.CONFLICT;  
  
    ErrorResponse error =  
        new ErrorResponse(  
            LocalDateTime.now(),  
            status.value(),  
            status.getReasonPhrase(),  
            ex.getMessage(),  
            Map.of()  
        );  
}
```

```
        );

        return ResponseEntity
            .status(status)
            .body(error);
    }
}
```

52. Ejemplo 409

```
{
  "timestamp": "2026-10-04T19:01:00",
  "status": 409,
  "error": "Conflict",
  "message": "Released animals cannot receive treatments",
  "details": {}
}
```

PARTE XI — VALIDATION HANDLER

53. MethodArgumentNotValidException

```
@ExceptionHandler(
    MethodArgumentNotValidException.class
)
public ResponseEntity<ErrorResponse>
handleValidation(
    MethodArgumentNotValidException ex) {

    Map<String, String> details =
        ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .collect(
                Collectors.toMap(
                    FieldError::getField,
                    FieldError::getDefaultMessage,
                    (first, second) -> first
                )
            );

    HttpStatus status =
        HttpStatus.BAD_REQUEST;

    ErrorResponse error =
        new ErrorResponse(
            LocalDateTime.now(),
            status.value(),
            status.getReasonPhrase(),
            "Request validation failed",
            details
        );
}
```

```
        );  
  
        return ResponseEntity  
            .status(status)  
            .body(error);  
    }  
}
```

54. Ejemplo de validación

```
{  
  "timestamp": "2026-10-04T19:01:00",  
  "status": 400,  
  "error": "Bad Request",  
  "message": "Request validation failed",  
  "details": {  
    "animalCode": "Animal code is required",  
    "specialistCode": "Specialist code is required",  
    "type": "Treatment type is required",  
    "description": "Description is required"  
  }  
}
```

PARTE XII — JSON INVÁLIDO

55. HttpResponseMessageNotReadableException

Ejemplo:

```
{  
  "status": "FLYING"  
}
```

FLYING no pertenece a `RescueStatus`.

Handler:

```
@ExceptionHandler(  
    HttpResponseMessageNotReadableException.class  
)  
public ResponseEntity<ErrorResponse>  
handleMessageNotReadable(  
    HttpResponseMessageNotReadableException ex) {  
  
    HttpStatus status =  
        HttpStatus.BAD_REQUEST;  
  
    ErrorResponse error =  
        new ErrorResponse(  

```

```
        LocalDateTime.now(),
        status.value(),
        status.getReasonPhrase(),
        "Malformed or invalid JSON request",
        Map.of(
            "body",
            "Check JSON syntax and enum values"
        )
    );

    return ResponseEntity
        .status(status)
        .body(error);
}
```

PARTE XIII — QUERY PARAMETER INVÁLIDO

56. MethodArgumentTypeMismatchException

Request:

```
GET /api/rescue-cases?status=FLYING
```

Handler:

```
@ExceptionHandler(
    MethodArgumentTypeMismatchException.class
)
public ResponseEntity<ErrorResponse>
handleTypeMismatch(
    MethodArgumentTypeMismatchException ex) {

    HttpStatus status =
        HttpStatus.BAD_REQUEST;

    Map<String, String> details =
        Map.of(
            ex.getName(),
            "Invalid value: "
            + String.valueOf(
                ex.getValue()
            )
        );

    ErrorResponse error =
        new ErrorResponse(
            LocalDateTime.now(),
            status.value(),
            status.getReasonPhrase(),
            "Invalid request parameter",
            details
        );
}
```

```
        return ResponseEntity
            .status(status)
            .body(error);
    }
```

PARTE XIV — ERROR INESPERADO

57. Exception

```
@ExceptionHandler(
    Exception.class
)
public ResponseEntity<ErrorResponse>
handleUnexpectedException(
    Exception ex) {

    HttpStatus status =
        HttpStatus.INTERNAL_SERVER_ERROR;

    ErrorResponse error =
        new ErrorResponse(
            LocalDateTime.now(),
            status.value(),
            status.getReasonPhrase(),
            "An unexpected error occurred",
            Map.of()
        );

    return ResponseEntity
        .status(status)
        .body(error);
}
```

No exponer al cliente:

```
stack trace
SQL
passwords
secrets
detalles internos
```

58. Resumen de handlers

Excepción	HTTP
MethodArgumentNotValidException	400
HttpMessageNotReadableException	400

Excepción	HTTP
MethodArgumentTypeMismatchException	400
ResourceNotFoundException	404
BusinessRuleException	409
Exception	500

Todos retornan:

```
ResponseEntity<ErrorResponse>
```

PARTE XV — TESTS DE CONTROLLER

59. Objetivo

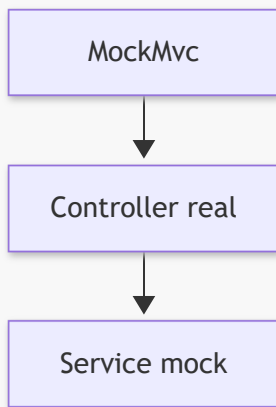
No queremos probar:

```
PostgreSQL  
Hibernate  
Repository  
implementación interna del Service
```

Queremos probar:

```
URL  
HTTP Method  
JSON  
Validation  
Status Code  
Response Body  
Exception Handling  
Delegación a Service
```

60. Arquitectura



61. @WebMvcTest

Ejemplo:

```
@WebMvcTest(  
    RescueCaseController.class  
)  
@Import(  
    GlobalExceptionHandler.class  
)  
class RescueCaseControllerTest {  
}
```

62. @MockitoBean

```
@MockitoBean  
private RescueCaseService service;
```

63. MockMvc

```
@Autowired  
private MockMvc mockMvc;
```

64. Estructura

```
@WebMvcTest(  
    RescueCaseController.class
```

```
)
@Import(
    GlobalExceptionHandler.class
)
class RescueCaseControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private RescueCaseService service;
}
```

PARTE XVI — TEST GET

65. Caso existente

```
@Test
void shouldReturnRescueCaseByCode()
    throws Exception {

    RescueCaseResponse response =
        new RescueCaseResponse(
            1L,
            "RES-2026-001",
            LocalDate.of(
                2026,
                8,
                20
            ),
            "Bahia Concha",
            RescueStatus
                .IN_REHABILITATION,
            "DB-CAR",
            "AN-2026-001"
        );

    when(
        service.findByCode(
            "RES-2026-001"
        )
    ).thenReturn(response);

    mockMvc.perform(
        get(
            "/api/rescue-cases/{code}",
            "RES-2026-001"
        )
    )
    .andExpect(
        status().isOk()
    )
    .andExpect(
        jsonPath("$.caseCode")
            .value("RES-2026-001")
    );
}
```

```
    )
    .andExpect(
        jsonPath("$.status")
            .value(
                "IN_REHABILITATION"
            )
    );

    verify(service)
        .findByCode(
            "RES-2026-001"
        );
}
```

66. Qué se está probando

```
GET correcto
URL correcta
200 OK
JSON correcto
Service invocado
```

No se prueba cómo Service encuentra el caso.

PARTE XVII — TEST 404

67. Recurso inexistente

```
@Test
void shouldReturn404WhenCaseDoesNotExist()
    throws Exception {

    when(
        service.findByCode("RES-999")
    ).thenThrow(
        new ResourceNotFoundException(
            "Rescue case not found: RES-999"
        )
    );

    mockMvc.perform(
        get(
            "/api/rescue-cases/{code}",
            "RES-999"
        )
    )
    .andExpect(
        status().isNotFound()
    )
    .andExpect(
```

```
        jsonPath("$.timestamp")
            .exists()
    )
    .andExpect(
        jsonPath("$.status")
            .value(404)
    )
    .andExpect(
        jsonPath("$.error")
            .value("Not Found")
    )
    .andExpect(
        jsonPath("$.message")
            .value(
                "Rescue case not found: RES-999"
            )
    )
    .andExpect(
        jsonPath("$.details")
            .isMap()
    );
}
```

PARTE XVIII — TEST REQUEST PARAM

68. Buscar por status

Request:

```
GET /api/rescue-cases?status=IN_REHABILITATION
```

Mock:

```
when(
    service.findByStatus(
        RescueStatus.IN_REHABILITATION
    )
)
.thenReturn(
    List.of(case1, case2)
);
```

Verificar:

```
200 OK
2 elementos
status IN_REHABILITATION
```

69. Status inválido

Request:

```
GET /api/rescue-cases?status=FLYING
```

Debe producir:

```
400 Bad Request
```

Assertions:

```
.andExpect(  
    jsonPath("$.status")  
        .value(400)  
)  
.andExpect(  
    jsonPath("$.error")  
        .value("Bad Request")  
)  
.andExpect(  
    jsonPath("$.message")  
        .value(  
            "Invalid request parameter"  
        )  
)  
.andExpect(  
    jsonPath("$.details.status")  
        .exists()  
)  
);
```

PARTE XIX — TEST PATCH

70. Cambiar status correctamente

```
@Test  
void shouldChangeRescueCaseStatus()  
    throws Exception {  
  
    RescueCaseResponse response =  
        /* construir response */;  
  
    when(  
        service.changeStatus(  
            eq("RES-001"),  
            any(  
                ChangeRescueStatusRequest.class  
            )  
        )  
    )
```

```
    )
    ).thenReturn(response);

    mockMvc.perform(
        patch(
            "/api/rescue-cases/{code}/status",
            "RES-001"
        )
        .contentType(
            MediaType.APPLICATION_JSON
        )
        .content("""
            {
                "status":
                    "READY_FOR_RELEASE"
            }
            """)
    )
    .andExpect(
        status().isOk()
    )
    .andExpect(
        jsonPath("$.status")
            .value(
                "READY_FOR_RELEASE"
            )
    )
    );

    verify(service)
        .changeStatus(
            eq("RES-001"),
            any(
                ChangeRescueStatusRequest.class
            )
        )
    );
}
```

71. Request inválido

Enviar:

```
{}
```

Esperado:

```
400 Bad Request
```

Assertions:

```
.andExpect(
    jsonPath("$.timestamp")
        .exists()
)
```

```
)  
.andExpect(  
    jsonPath("$.status")  
        .value(400)  
)  
.andExpect(  
    jsonPath("$.error")  
        .value("Bad Request")  
)  
.andExpect(  
    jsonPath("$.message")  
        .value(  
            "Request validation failed"  
        )  
)  
.andExpect(  
    jsonPath("$.details.status")  
        .value(  
            "Status is required"  
        )  
)  
);
```

Además:

```
verify(  
    service,  
    never()  
)  
.changeStatus(  
    anyString(),  
    any()  
)  
);
```

72. BusinessException

Configurar:

```
when(  
    service.changeStatus(  
        eq("RES-001"),  
        any()  
    )  
)  
.thenThrow(  
    new BusinessException(  
        "Invalid status transition"  
    )  
)  
);
```

Esperado:

409 Conflict

Assertions:

```
.andExpect(  
    jsonPath("$.status")  
        .value(409)  
)  
.andExpect(  
    jsonPath("$.error")  
        .value("Conflict")  
)  
.andExpect(  
    jsonPath("$.message")  
        .value(  
            "Invalid status transition"  
        )  
)  
.andExpect(  
    jsonPath("$.details")  
        .isMap()  
)  
);
```

PARTE XX — TREATMENT CONTROLLER TEST

73. Estructura

```
@WebMvcTest(  
    TreatmentController.class  
)  
@Import(  
    GlobalExceptionHandler.class  
)  
class TreatmentControllerTest {  
  
    @Autowired  
    private MockMvc mockMvc;  
  
    @MockitoBean  
    private TreatmentService service;  
}
```

74. POST exitoso

Enviar:

```
{
  "animalCode": "AN-001",
  "specialistCode": "SPEC-001",
  "performedAt": "2026-08-21T09:00:00",
  "type": "WOUND_CARE",
  "description": "Cleaning and treatment of flipper injury."
}
```

Esperado:

201 Created

Verificar:

```
verify(service)
  .register(
    any(
      CreateTreatmentRequest.class
    )
  );
```

75. POST inválido

Enviar:

```
{
  "animalCode": "",
  "specialistCode": "",
  "type": null,
  "description": ""
}
```

Esperado:

400 Bad Request

Verificar:

```
verify(
  service,
  never()
)
.register(any());
```

76. Animal inexistente

Service:

```
throw new ResourceNotFoundException(  
    "Animal not found: AN-999"  
);
```

Respuesta:

```
{  
  "status": 404,  
  "error": "Not Found",  
  "message": "Animal not found: AN-999",  
  "details": {}  
}
```

77. BusinessException

Ejemplo:

```
Released animals cannot receive treatments
```

Respuesta:

```
409 Conflict
```

PARTE XXI — ANIMAL CONTROLLER TEST

78. Estructura

```
@WebMvcTest(  
    AnimalController.class  
)  
@Import(  
    GlobalExceptionHandler.class  
)  
class AnimalControllerTest {  
  
    @Autowired  
    private MockMvc mockMvc;  
  
    @MockitoBean
```

```
private AnimalService animalService;

@MockitoBean
private TreatmentService treatmentService;
}
```

79. Test findByCode()

Request:

```
GET /api/animals/AN-001
```

Esperado:

```
200 OK
```

Verificar:

```
verify(animalService)
    .findByCode("AN-001");
```

80. Test findAnimalsInRehabilitation()

Request:

```
GET /api/animals/in-rehabilitation
```

Mock:

```
when(
    animalService
        .findAnimalsInRehabilitation()
)
.thenReturn(
    List.of(animal1, animal2)
);
```

Esperado:

```
200 OK
2 elementos
```

Verificar:

```
verify(animalService)
    .findAnimalsInRehabilitation();
```

81. Test tratamientos

Request:

```
GET /api/animals/AN-001/treatments
```

Verificar:

```
verify(treatmentService)
    .findByAnimalCode(
        "AN-001"
    );
```

82. Test eligibility

Request:

```
GET /api/animals/AN-001/treatment-eligibility
```

Mock:

```
when(
    animalService
        .canReceiveTreatment(
            "AN-001"
        )
)
    .thenReturn(true);
```

Respuesta:

```
{
  "animalCode": "AN-001",
  "eligible": true
}
```

Verificar:

```
verify(animalService)
    .canReceiveTreatment(
        "AN-001"
    );
```

PARTE XXII — RETO INTEGRADOR

83. Escenario

Caso:

```
RES-2026-100
status = IN_REHABILITATION
```

Animal:

```
AN-2026-100
Green Sea Turtle
```

Especialista:

```
SPEC-001
Elena Vargas
active = true
```

84. Operación 1

```
GET /api/rescue-cases/RES-2026-100
```

Resultado:

```
200 OK
```

85. Operación 2

```
GET /api/animals/AN-2026-100
```

Resultado:

200 OK

86. Operación 3

POST /api/treatments

```
{
  "animalCode": "AN-2026-100",
  "specialistCode": "SPEC-001",
  "performedAt": "2026-08-21T09:00:00",
  "type": "WOUND_CARE",
  "description": "Cleaning of left front flipper injury."
}
```

Resultado:

201 Created

87. Operación 4

Cambiar:

```
IN_REHABILITATION
  ↓
READY_FOR_RELEASE
```

Request:

PATCH /api/rescue-cases/RES-2026-100/status

Resultado:

200 OK

88. Operación 5

Enviar:

```
{  
  "status": null  
}
```

Resultado:

400 Bad Request

con `ErrorResponse`.

89. Operación 6

Intentar transición inválida.

Resultado:

409 Conflict

90. Operación 7

```
GET /api/animals/AN-999
```

Resultado:

404 Not Found

PARTE XXIII — RETO DEL ESTUDIANTE

91. Elegibilidad

Implementar completamente:

```
GET /api/animals/{animalCode}/treatment-eligibility
```

Respuesta:

```
{  
  "animalCode": "AN-2026-100",  
  "eligible": true  
}
```

Debe incluir:

1. DTO.
2. Controller endpoint.
3. llamada a `AnimalService`.
4. test `200`.
5. test `404`.
6. `verify(...)`.

PARTE XXIV — DISEÑO REST

92. Analizar endpoints

¿Cuál comunica mejor un recurso?

```
GET /api/getAnimal?id=1
```

o:

```
GET /api/animals/AN-001
```

Justifica.

93. Analizar PATCH

¿Cuál expresa mejor una modificación parcial?

```
POST /changeStatus
```

o:

```
PATCH /api/rescue-cases/RES-001/status
```

Justifica.

94. Recursos anidados

Compara:

```
GET /api/animals/AN-001/treatments
```

con:

```
GET /api/treatments?animal=AN-001
```

Ambos pueden ser válidos.

Explica qué relación comunica cada uno.

PARTE XXV — ANTI-PATRONES

95. Controller → Repository

Evitar:

```
@RestController
public class AnimalController {

    private AnimalRepository repository;
}
```

Preferir:

```
Controller
  ↓
Service
  ↓
Repository
```

96. Reglas de negocio en Controller

Evitar:

```
if (
    rescueCase.getStatus()
        == RescueStatus.RELEASED
) {
    ...
}
```

Pertenece a:

Service

97. Retornar Entity

Evitar:

```
@GetMapping(...)
public Animal find(...) {
}
```

Preferir:

```
public AnimalResponse find(...) {
}
```

98. try/catch en cada endpoint

Evitar duplicar:

```
try {
    ...
}
catch (...) {
    ...
}
```

Preferir:

```
@RestControllerAdvice
```

99. 200 para todo

No responder siempre:

```
200 OK
```

Distinguir:

200
201
400
404
409
500

PARTE XXVI — RESPONSABILIDADES

100. Clasificar

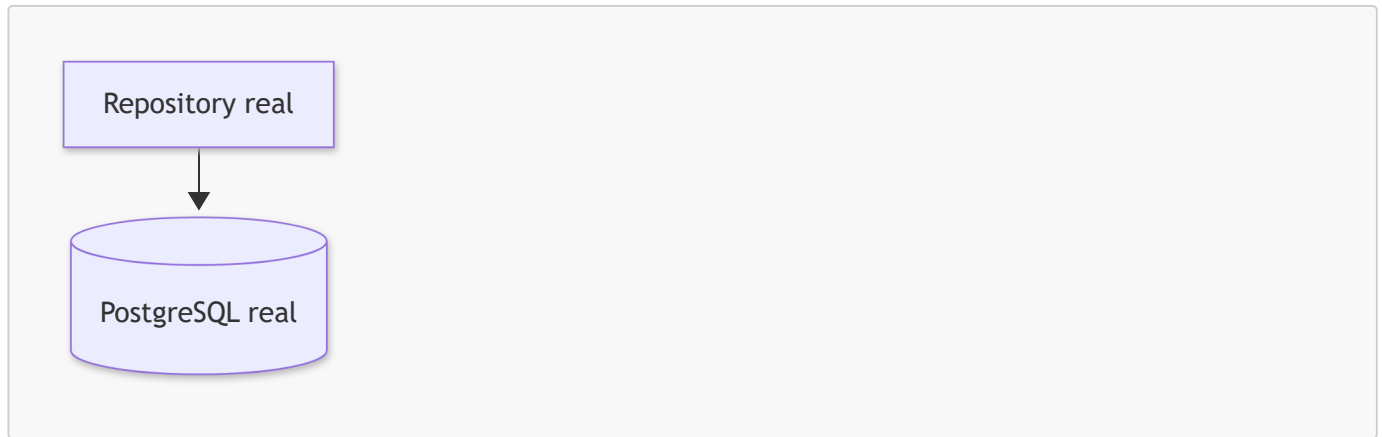
Necesidad	Capa
Recibir JSON	_____
Verificar @NotBlank	_____
Buscar Animal	_____
Verificar especialista activo	_____
Ejecutar query	_____
Convertir excepción en 409	_____
Abrir transacción	_____
Retornar 201	_____

Opciones:

Controller
DTO Validation
Service
Repository
ControllerAdvice

PARTE XXVII — TESTING POR CAPA

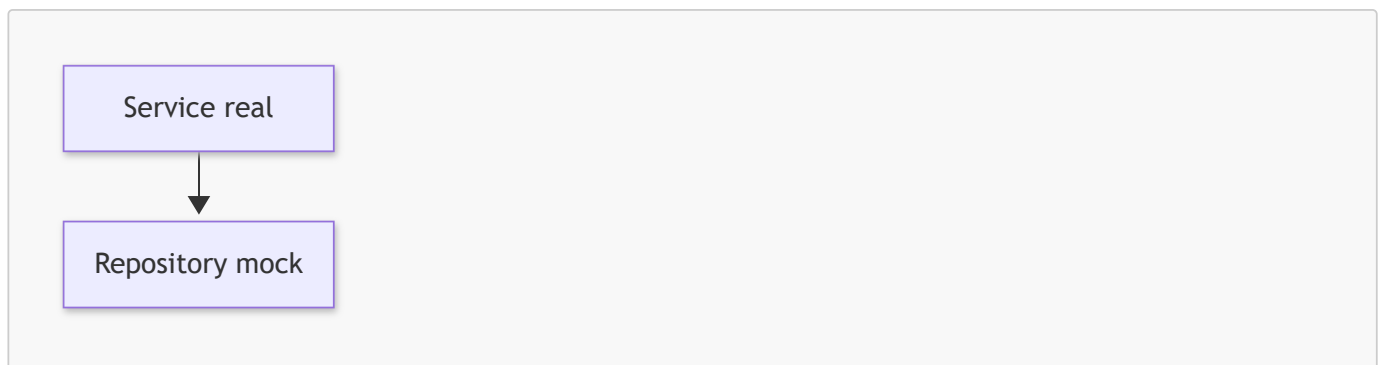
101. Repository Test



Herramienta:

Testcontainers

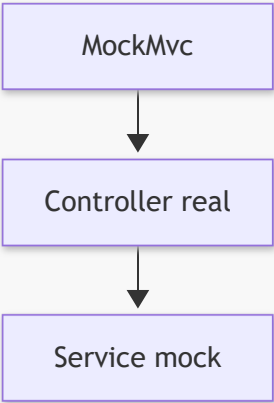
102. Service Test



Herramienta:

Mockito

103. Controller Test



Herramientas:

```
@WebMvcTest
@MockitoBean
MockMvc
```

104. Cada prueba tiene un objetivo

- Repository test
→ persistencia
- Service test
→ reglas de negocio
- Controller test
→ contrato HTTP

PARTE XXVIII — MATRIZ FINAL DE TRAZABILIDAD

105. Service → Controller → Test

Método Service	Controller	Test
RescueCaseService.findByCode()	RescueCaseController	shouldReturnRescueCaseByCode
RescueCaseService.findByStatus()	RescueCaseController	shouldReturnCasesByStatus
RescueCaseService.changeStatus()	RescueCaseController	shouldChangeStatus
TreatmentService.register()	TreatmentController	shouldCreateTreatment
TreatmentService.findByAnimalCode()	AnimalController	shouldReturnAnimalTreatments
AnimalService.findByCode()	AnimalController	shouldReturnAnimalByCode

Método Service	Controller	Test
AnimalService.findAnimalsInRehabilitation()	AnimalController	shouldReturnAnimalsInRehabilitation
AnimalService.canReceiveTreatment()	AnimalController	shouldReturnTreatmentEligibility

PARTE XXIX — CASOS MÍNIMOS DE ERROR

106. Tabla

Situación	HTTP	Handler
DTO inválido	400	MethodArgumentNotValidException
JSON inválido	400	HttpMessageNotReadableException
Query parameter inválido	400	MethodArgumentTypeMismatchException
Recurso inexistente	404	ResourceNotFoundException
Regla de negocio	409	BusinessRuleException
Error inesperado	500	Exception

Todos retornan:

```
ResponseEntity<ErrorResponse>
```

PARTE XXX — TESTS MÍNIMOS

107. Casos obligatorios

1. GET RescueCase existente
→ 200
2. GET RescueCase inexistente
→ 404 + ErrorResponse
3. GET RescueCases por status
→ 200
4. GET status inválido
→ 400 + ErrorResponse
5. PATCH status válido
→ 200
6. PATCH request inválido
→ 400 + details
7. PATCH transición inválida
→ 409 + ErrorResponse

8. POST Treatment válido
→ 201
9. POST Treatment inválido
→ 400 + details
10. POST animal inexistente
→ 404
11. POST regla de negocio
→ 409
12. GET Animal
→ 200
13. GET animales en rehabilitación
→ 200
14. GET tratamientos del animal
→ 200
15. GET eligibility
→ 200
16. GET Animal inexistente
→ 404 + ErrorResponse
17. JSON enum inválido
→ 400
18. Error inesperado
→ 500

PARTE XXXI — EJECUCIÓN

108. Ejecutar

```
mvn clean test
```

Resultado esperado:

```
BUILD SUCCESS
```

Los tests de Controller no necesitan:

```
PostgreSQL  
Testcontainers  
Repository real
```

PARTE XXXII — CHECKLIST

109. Implementación

```
[ ] spring-boot-starter-webmvc
[ ] spring-boot-starter-validation
[ ] RescueCaseController
[ ] TreatmentController
[ ] AnimalController
[ ] @RestController
[ ] @RequestMapping
[ ] @GetMapping
[ ] @PostMapping
[ ] @PatchMapping
[ ] @PathVariable
[ ] @RequestParam
[ ] @RequestBody
[ ] @Valid
[ ] Bean Validation
[ ] ResponseEntity
```

110. Error handling

```
[ ] ErrorResponse
[ ] timestamp
[ ] status
[ ] error
[ ] message
[ ] details
[ ] @RestControllerAdvice
```

```
[ ] ResponseEntity<ErrorResponse>

[ ] validation → 400

[ ] invalid JSON → 400

[ ] invalid query parameter → 400

[ ] not found → 404

[ ] business rule → 409

[ ] unexpected → 500
```

111. Cobertura Service

```
[ ] RescueCaseService.findByCode()

[ ] RescueCaseService.findByStatus()

[ ] RescueCaseService.changeStatus()

[ ] TreatmentService.register()

[ ] TreatmentService.findByAnimalCode()

[ ] AnimalService.findByCode()

[ ] AnimalService.findAnimalsInRehabilitation()

[ ] AnimalService.canReceiveTreatment()
```

112. Testing

```
[ ] @WebMvcTest

[ ] @MockitoBean

[ ] MockMvc

[ ] jsonPath

[ ] verify(...)

[ ] verify(..., never())

[ ] ErrorResponse verificado

[ ] details verificado
```

[] todos los métodos Service cubiertos

PARTE XXXIII — PREGUNTAS DE SUSTENTACIÓN

113. Preguntas

1. ¿Cuál es la responsabilidad del Controller?
2. ¿Qué diferencia existe entre Controller y Service?
3. ¿Por qué Controller no debería usar Repository directamente?
4. ¿Qué hace `@RestController`?
5. ¿Qué hace `@RequestMapping`?
6. ¿Qué diferencia existe entre `@PathVariable` y `@RequestParam`?
7. ¿Qué hace `@RequestBody`?
8. ¿Qué hace `@Valid`?
9. ¿Qué diferencia existe entre validación de entrada y regla de negocio?
10. ¿Cuándo utilizar GET?
11. ¿Cuándo utilizar POST?
12. ¿Cuándo utilizar PATCH?
13. ¿Qué significa 200?
14. ¿Qué significa 201?
15. ¿Qué significa 400?
16. ¿Qué significa 404?
17. ¿Qué significa 409?
18. ¿Qué significa 500?
19. ¿Para qué sirve `ResponseEntity`?
20. ¿Qué problema resuelve `@RestControllerAdvice`?
21. ¿Por qué conviene tener un `ErrorResponse` común?
22. ¿Para qué sirve `details`?
23. ¿Qué diferencia existe entre `MethodArgumentNotValidException` y `BusinessRuleException`?
24. ¿Qué prueba `@WebMvcTest`?
25. ¿Por qué utilizamos un Service mock?
26. ¿Qué permite probar `MockMvc`?
27. ¿Por qué el test de Controller no necesita PostgreSQL?
28. ¿Por qué `verify(..., never())` es útil en validaciones?
29. ¿Qué capa debe abrir transacciones?
30. ¿Qué capa debe decidir una transición de estado?

PARTE XXXIV — COMPETENCIA FINAL

114. Flujo exitoso

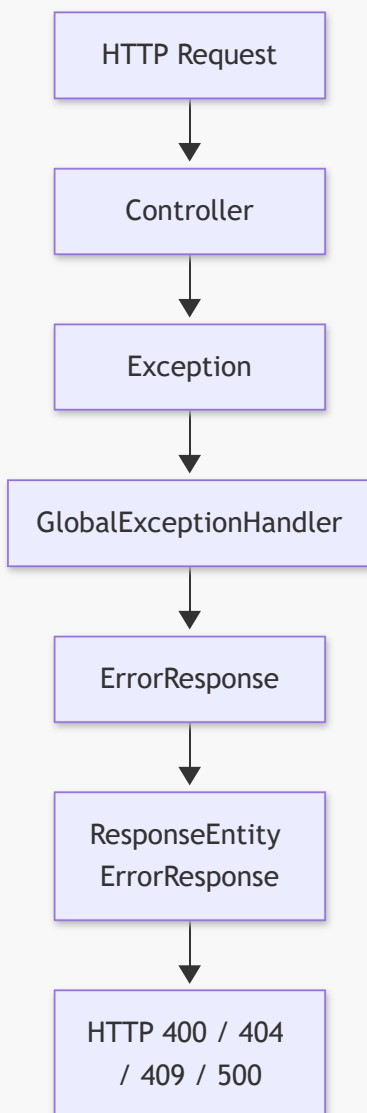


Syntax error in text

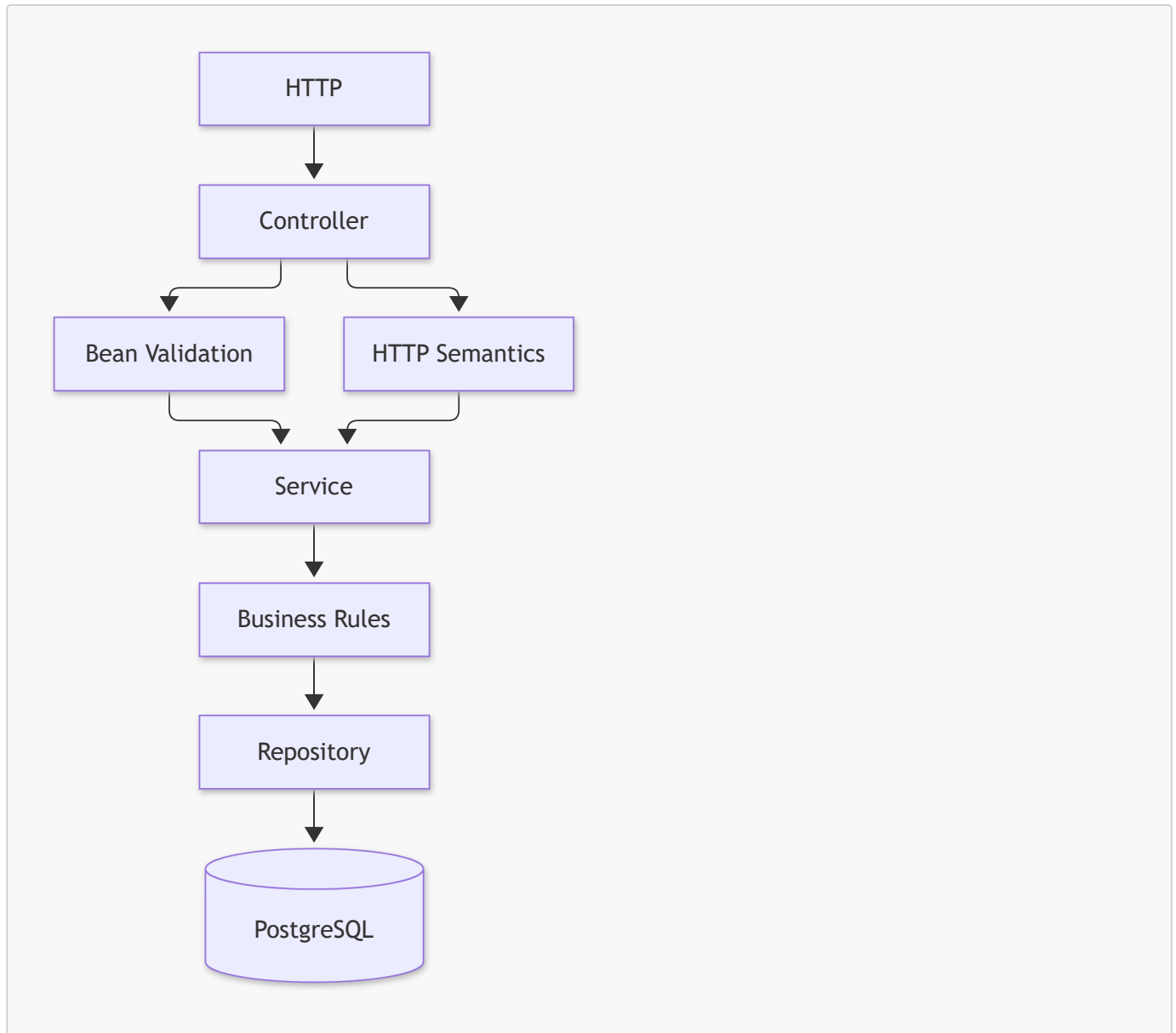
mermaid version 12.1.0

```
Parse error on line 3: ...B[Controller]B --> C[@Valid]C --> D[Re
-----^ Expecting 'AMP', 'COLON', 'PIPE', 'TESTSTR',
'DOWN', 'DEFAULT', 'NUM', 'COMMA', 'NODE_STRING', 'BRKT', 'MINUS', 'MULT',
'UNICODE_TEXT', got 'LINK_ID'
```

115. Flujo con error



116. Arquitectura final



117. Idea principal

El estudiante debe comprender:

El Controller traduce HTTP hacia la aplicación; el Service decide qué está permitido por el negocio.

Y para errores:

`GlobalExceptionHandler` traduce excepciones de aplicación a un contrato HTTP consistente mediante `ResponseEntity<ErrorResponse>`.

Finalmente, debe quedar claro que:

```
Controller
  ↓
NO implementa negocio

Service
  ↓
Sí implementa negocio
```

Repository



Sí accede a persistencia

Ese modelo mental es más importante que memorizar anotaciones.